User Acceptance Testing (UAT) Template

Date	30 September 2026
Team ID	SWTID-2026-2365
Project Name	AI FitTrack API – Fitness Tracking API with Google Gemini AI
Maximum Marks	

Project Overview:

Project Name: AI FitTrack API

Project Description: A RESTful fitness tracking API built with Node.js, Express.js and MongoDB. Users can register, log in with JWT authentication, manage and search their workouts, and get AI-generated workout recommendations and fitness insights from Google Gemini.

Project Version: 1.0.0

Testing Period: 21 September 2026 to 26 September 2026

Testing Scope:

Features and functionalities to be tested:

1. User authentication – registration, login and profile (JWT protected).

2. Workout management – add, view all, view by ID, update and delete workouts.

3. Workout search – by name, category and date.

4. AI assistance – workout recommendation and fitness insights using Google Gemini.

User stories / requirements to be tested: USN-3 to USN-17 of the Product Backlog (registration, login, profile, workout CRUD, data privacy, search and AI features).

Testing Environment:

URL/Location: http://localhost:5000 (API base URL; endpoints under /api/auth, /api/workouts and /api/ai). Test using the FitTrack Postman collection.

Credentials (if required): john@gmail.com / 123456 (test user registered through the API; the JWT token is sent as a Bearer token)

Test Cases:

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Pass/ Fail
TC-001	Register a new user with valid details	Step 1: POST /api/auth/register	201 Created; response has success = true, a JWT token		

		Step 2: Send name, email and a password of 6+ characters	and the user id, name and email.		
TC-002	Register with an email that already exists	Step 1: POST /api/auth/register Step 2: Send the same email used earlier	400 error message: Email already registered.		
TC-003	Register with missing fields (negative)	Step 1: POST /api/auth/register Step 2: Send only email and password	400 error message asking for name, email and password.		
TC-004	Login with valid credentials	Step 1: POST /api/auth/login Step 2: Send registered email and password	200 OK; success = true and a JWT token is returned.		
TC-005	Login with wrong password (negative)	Step 1: POST /api/auth/login Step 2: Send a registered email with a wrong password	401 error message: Invalid credentials.		
TC-006	View profile with a valid token	Step 1: Login and copy the token Step 2: GET /api/auth/profile with Bearer token	200 OK; logged-in user's id, name and email are returned.		
TC-007	Access a protected route without a token (negative)	Step 1: GET /api/workouts without an Authorization header	Request is rejected as unauthorised; no workout data is returned.		
TC-008	Add a workout with valid details	Step 1: POST /api/workouts with workoutName, category, duration, caloriesBurned and workoutDate	201 Created; the saved workout with its _id is returned.		
TC-009	Add a workout with an unsupported	Step 1: POST /api/workouts with category = Swimming	Validation error: category is not supported; workout is not saved.		

	category (negative)				
TC-010	View all workouts	Step 1: GET /api/workouts with Bearer token	200 OK; count and only the user's workouts, latest date first.		
TC-011	View a workout by ID	Step 1: GET /api/workouts/:id using a saved workout id	200 OK; the matching workout is returned.		
TC-012	Access another user's workout (negative)	Step 1: Login as a second user Step 2: GET /api/workouts/:id of the first user's workout	404 error message: Workout not found.		
TC-013	Update a workout	Step 1: PUT /api/workouts/:id with new workoutName, duration and caloriesBurned	200 OK; the workout is returned with the updated values.		
TC-014	Delete a workout	Step 1: DELETE /api/workouts/:id Step 2: GET the same id again	200 OK: Workout removed successfully; the second request returns 404.		
TC-015	Search workouts by name or category	Step 1: GET /api/workouts/search?q=running	200 OK; only workouts whose name or category matches the keyword (any letter case).		
TC-016	Search workouts by date	Step 1: GET /api/workouts/search?q=2026-06-26	200 OK; workouts logged on that date are returned.		
TC-017	Search without a search term (negative)	Step 1: GET /api/workouts/search with no q parameter	400 error message asking for the q query parameter.		
TC-018	AI workout recommendation	Step 1: POST /api/ai/workout-recommendation with age, fitnessGoal and experience	200 OK; a recommendation text generated by Gemini is returned.		

TC-019	AI recommendation with missing fields (negative)	Step 1: POST /api/ai/workout-recommendation with only age	400 error message asking for age, fitnessGoal and experience.		
TC-020	AI fitness insights	Step 1: POST /api/ai/fitness-insights with totalWorkouts, averageDuration and totalCaloriesBurned	200 OK; an insight text generated by Gemini is returned.		
TC-021	AI insights with missing fields (negative)	Step 1: POST /api/ai/fitness-insights with only totalWorkouts	400 error message asking for all three values.		

Bug Tracking:

Bug ID	**Bug Description**	**Steps to reproduce**	**Severity**	**Status**	**Additional feedback**
BG-001	Search fails on special characters	Step 1: Login as a user Step 2: GET /api/workouts/search?q=(Step 3: Observe the response	Medium	Open	Search text is used directly as a regular expression and returns a server error. Escape the input.
BG-002	A workout's owner can be changed by an update request	Step 1: Login and pick one of your workouts Step 2: PUT /api/workouts/:id with body {"user": "<another user id>"}	High	Open	Update accepts the whole request body. Allow only workoutName, category, duration, caloriesBurned and workoutDate.
BG-003	Zero calories or duration rejected as missing	Step 1: POST /api/workouts with caloriesBurned = 0	Low	Open	Validation treats 0 as empty although the schema allows 0 calories. Check for undefined instead.

BG-004	Valid email addresses with long domain endings rejected	Step 1: POST /api/auth/register with email john@example.info	Low	Open	Email pattern accepts only 2–3 letter endings. Use a more flexible pattern.
BG-005	Fitness insights ignore the configured Gemini model	Step 1: Set GEMINI_MODEL in .env to a different model Step 2: POST /api/ai/fitness-insights	Low	Open	Insights use a fixed gemini-2.5-flash while recommendations use GEMINI_MODEL. Use the environment value in both.

Sign-off:

Tester Name: Srinivasan K, Dillibabu J, Akash S, Mohanasundaram G, Jagadeesh S (Team SWTID-2026-2365)

Date: 26 September 2026

Signature: _____

Notes:

- Ensure that all test cases cover both positive and negative scenarios.
- Encourage testers to provide detailed feedback, including any suggestions for improvement.
- Bug tracking should include details such as severity, status, and steps to reproduce.
- Obtain sign-off from both the project manager and product owner before proceeding with deployment.